Material complementario · Lenguajes de programación



CONOCE EL LENGUAJE

JavaScript

el lenguaje de la **web**

Nació para el navegador
y hoy corre en todas partes.

CURSO
Desarrollando con Inteligencia Artificial

INSTRUCTOR
Felipe Cuevas C.

ANTES DE EMPEZAR

Ficha técnica

Todo lo esencial de JavaScript en una sola tarjeta. Si solo tienes un minuto, quédate con esta página.

LENGUAJE DE PROGRAMACIÓN

JavaScript

Estándar oficial: ECMAScript (ECMA-262)

Desde 1995

CREADOR
Brendan Eich, en Netscape

MANTIENE HOY
Comité TC39 de Ecma International (navegadores, empresas y comunidad)

PARADIGMAS
Orientado a eventos · Funcional · Orientado a objetos (prototipos) · Asíncrono

TIPADO
Dinámico y débil (TypeScript le agrega tipos)

EJECUCIÓN
Interpretado y optimizado con JIT por motores como V8

MEMORIA
Automática con *garbage collector*

VERSIÓN ACTUAL (NOV. 2026)
ECMAScript 2026
Aprobada el 30 de junio de 2026. En servidores: Node.js 26, versión LTS desde octubre de 2026

ARCHIVOS TÍPICOS
`.js` `.mjs` `.jsx` `.ts` `package.json`

DÓNDE CORRE
Navegadores · Servidores (Node.js, Deno, Bun) · Móvil (React Native) · Escritorio (Electron) · Nube y *serverless*

DATO CURIOSO

El primer prototipo se escribió en **10 días**. Se llamó Mocha, luego LiveScript y finalmente JavaScript. **No tiene relación con Java**: el nombre fue una estrategia de marketing de la época.

¿JAVASCRIPT O ECMASCRIPT?

ECMAScript es el nombre del estándar oficial; JavaScript es el nombre que usamos todos. Cada año sale una nueva edición, como ES2026.

1 INTRODUCCIÓN

De dónde viene

De darle vida a las páginas web a correr en servidores, celulares y la nube.

JavaScript nació en 1995 en **Netscape**, la empresa del navegador más popular de la época. **Brendan Eich** creó su primer prototipo en apenas 10 días, con una meta clara: darle vida a las páginas web, validar formularios y reaccionar a los clics. En 1997 se estandarizó como **ECMAScript**, para que todos los navegadores lo hablaran igual. En 2009, **Node.js** lo sacó del navegador y lo llevó a los servidores, y en 2015 la versión **ES6** lo modernizó por completo. Hoy es el único lenguaje que los navegadores entienden de forma nativa.

HITO: PRESENTE EN CASI TODA LA WEB

98,9 %

de los sitios web usan JavaScript (W3Techs, nov. 2026). Además, fue Lenguaje del Año TIOBE 2014 y el más usado en la encuesta de Stack Overflow 9 años seguidos (2013–2021).

¿Y en TIOBE, octubre de 2026?
Está en el **6.º lugar** (2,77 %). TIOBE mide búsquedas en internet, no uso real: por eso un lenguaje omnipresente no aparece primero.

Línea de tiempo

1995 — Netscape crea JavaScript

1997 — Primer estándar ECMAScript

2009 — Node.js lo lleva al servidor

2012 — Microsoft lanza TypeScript

2015 — ES6: el JavaScript moderno

2026 — ES2026 y Node.js 26 LTS

> **Si lo ves moverse en una página web, JavaScript está detrás.**
> Menús, formularios, mapas, chats en vivo: casi toda la interactividad de la web es JavaScript.

ACLAREMOS ALGO CLAVE

JavaScript, el motor y el entorno no son lo mismo

JavaScript es el lenguaje. Un motor lo ejecuta. Y el entorno decide qué puede hacer tu código.

CAPA 3 · EL ENTORNO DE EJECUCIÓN

Navegador · Node.js · Deno · Bun

El navegador te da la página (`document`); Node.js te da archivos, red y servidor (`fs`, `http`).

CAPA 2 · EL MOTOR

V8 · SpiderMonkey · JavaScriptCore

Leen y ejecutan tu código. V8 (de Google) está en Chrome, Edge y Node.js.

CAPA 1 · EL IDIOMA

JavaScript (ECMAScript)

Las reglas y palabras con que escribes: `const`, `function`, `async`, `import`…

¿Y TypeScript?



> **Un mismo idioma, muchos escenarios.**
>
> El lenguaje es igual en todos lados; lo que cambia son las herramientas que te presta cada entorno.

¿POR QUÉ IMPORTA?

Código que funciona en el navegador puede fallar en Node.js (y al revés). Cuando pidas ayuda a una IA, di **dónde corre tu código** (“navegador” o “Node.js 24”) y si usas JavaScript o TypeScript.

2 ESTRUCTURA

Cómo funciona

JavaScript se interpreta y se optimiza mientras corre. Y tiene un truco especial: el *event loop*.

¿Interpretado o compilado? Ambos, al vuelo

No compilas antes de ejecutar: el motor recibe el código fuente, lo convierte en *bytecode* y empieza a ejecutarlo de inmediato. Mientras corre, detecta las funciones que más se usan y las **compila a código de máquina** con su compilador **JIT**. Por eso JavaScript moderno es sorprendentemente rápido.

DENTRO DEL MOTOR V8

📄 app.js

▼ análisis del código

📦 *bytecode* + intérprete

▼ JIT para el código frecuente

🖥️ Código de máquina

EL EVENT LOOP

⏳ Pides datos a una API

▼ no esperas: sigues trabajando

📬 La respuesta llega a una cola

▼ el *event loop* la toma

✅ Se ejecuta tu `await`

> **Un solo hilo, pero nunca se queda esperando.**
>
> Gracias al *event loop*, un servidor Node.js atiende miles de conexiones sin bloquearse.

¿Y la memoria?

Garbage collector

El motor libera solo los objetos que ya no se usan. No necesitas liberar memoria a mano.

Cuidado con las fugas

Eventos que nunca se eliminan o intervalos que no se detienen mantienen objetos vivos y causan fugas de memoria.

ATENCIÓN A ESTO

Su tipado es débil: `"5" + 1` da `"51"`, pero `"5" - 1` da 4. Usa siempre `===` en lugar de `==` para comparar sin conversiones sorpresa.

2 ESTRUCTURA

Paradigmas, organización y versión actual

Paradigmas que soporta

Orientado a eventos
Clics, teclas, mensajes

Funcional
Funciones como valores

Objetos
Prototipos y `class`

Asíncrono
Promesas y `async/await`

La palabra `class` existe desde 2015, pero por dentro JavaScript usa **prototipos**: cada objeto hereda directamente de otro objeto.

Cómo se organiza el código: módulos

```
// archivo: calculadora.js

// lo público: se exporta
export function sumar(a, b) {
  return a + b;
}

// lo interno: no se exporta
function doble(x) {
  return x * 2;
}

// en otro archivo:
// import { sumar } from "./calculadora.js";
```

Módulo = un archivo que decide qué compartir con `export`.

import trae lo que otro módulo compartió.

Lo que no exportas es la parte interna del módulo. Los proyectos se describen en `package.json`.

Lo nuevo: una edición cada junio

ES2025 · jun. 2025
Ayudantes para iteradores, nuevos métodos de `Set` (unión, intersección) e importación de JSON.

Node.js 26 · LTS
Activa por defecto **Temporal**, la nueva API para manejar fechas y zonas horarias.

ES2026 · jun. 2026 (actual)
`Math.sumPrecise`, `Error.isError`, `Array.fromAsync` y conversión a base64.

Quién lo mantiene
El comité TC39 de Ecma International: Google, Mozilla, Apple, Microsoft y la comunidad.

MANOS AL CÓDIGO

Primer contacto

Un programa que filtra una lista y reacciona a un clic, siguiendo buenas prácticas. Lee los números de la derecha mientras recorres el código.

```
const cursos = [                          // ①
  { nombre: "JavaScript", horas: 30 },
  { nombre: "Python", horas: 24 },
  { nombre: "Java", horas: 40 },
];

const largos = cursos
  .filter((c) => c.horas >= 30)           // ②
  .map((c) => c.nombre);

for (const nombre of largos) {
  console.log(`Curso largo: ${nombre}`); // ③
}

// en el navegador: reaccionar a un clic
document.querySelector("#hola")           // ④
  .addEventListener("click", () => {
    alert("¡Hola!");
  });
```

1 const y objetos

`const` para valores que no se reasignan; `{ }` crea objetos al instante.

2 Funciones flecha

`(c) => …` es una función corta, ideal con `filter` y `map`.

3 Plantillas de texto

Con comillas invertidas (`` ` ``) insertas variables con `${ }`.

4 Eventos

Tu código espera a que el usuario haga algo y reacciona.

SALIDA EN LA CONSOLA

```
Curso largo: JavaScript
Curso largo: Java
```

> **No necesitas instalar nada para empezar.**
>
> Abre tu navegador, presiona F12 y escribe JavaScript en la consola: es el laboratorio más accesible del mundo.

3 PRINCIPALES USOS

Dónde brilla

Con un solo lenguaje puedes construir la interfaz, el servidor, la app móvil y la de escritorio.

Frontend web
Interfaces interactivas en cualquier navegador.

Backend y APIs
Servidores con Node.js, Deno o Bun.

Tiempo real
Chats, notificaciones, colaboración y juegos en línea.

Apps móviles
Android e iOS con React Native.

Apps de escritorio
Windows, macOS y Linux con Electron.

Nube y *serverless*
Funciones que se ejecutan bajo demanda en la nube.

Casos reales

Visual Studio Code
Microsoft

El editor de código más popular está construido con **Electron** y TypeScript: es, en esencia, una app web de escritorio.

React
Creado por Meta

Nació para las interfaces de Facebook y hoy es la librería de interfaces más usada del mundo, también en apps móviles con **React Native**.

PayPal
Pagos en línea

Fue uno de los primeros gigantes en migrar su web de Java a **Node.js**, unificando frontend y backend en un solo lenguaje.

PARA TENER EN CUENTA

Esta versatilidad dio origen al perfil **full stack JavaScript**: una misma persona puede programar la interfaz, el servidor y la base de datos sin cambiar de lenguaje.

4 PARA QUÉ SE RECOMIENDA

¿Cuándo elegirlo?

Ningún lenguaje sirve para todo. Este semáforo te ayuda a decidir con criterio.

Frontend web	🟢 **Ideal**	Es el único lenguaje que el navegador ejecuta de forma nativa. No hay alternativa real.
Apps web full stack	🟢 **Ideal**	Next.js y similares permiten frontend y backend en un mismo proyecto.
Tiempo real y APIs	🟢 **Ideal**	El *event loop* de Node.js maneja muy bien muchas conexiones simultáneas.
Integrar IA en una app	🟢 **Muy bueno**	SDK oficial de Anthropic (TypeScript), SDK oficial de MCP y AI SDK de Vercel para interfaces con IA.
Apps móviles	🟢 **Muy bueno**	React Native comparte código entre Android e iOS.
Escritorio	🟢 **Muy bueno**	Electron es productivo, aunque sus apps consumen más memoria que las nativas.
Microservicios y nube	🟢 **Muy bueno**	Arranque rápido, ideal para *serverless*; para cálculo pesado, otros lenguajes rinden más.
Machine learning	🟡 **Posible**	TensorFlow.js y Transformers.js ejecutan modelos en el navegador, pero el entrenamiento vive en Python.
Big data	🔴 **No recomendado**	Spark, Kafka y Hadoop pertenecen al mundo JVM y Python.
Cálculo intensivo y bajo nivel	🔴 **No recomendado**	Simulaciones, drivers o videojuegos exigentes: mejor C, C++ o Rust.

> **Elige JavaScript cuando tu producto viva en la web.**
>
> Y considera TypeScript desde el inicio: los mismos beneficios, con menos errores en proyectos grandes.

5 ECOSISTEMA

Tu caja de herramientas

npm es el registro de paquetes más grande del mundo. Estas son las piezas que más vas a encontrar.

Frontend

React · Interfaces con componentes reutilizables; la más usada.

Vue y Angular · Alternativas completas: Vue es simple; Angular, muy estructurado.

Svelte · Componentes que se compilan a JavaScript liviano.

Full stack

Next.js · Framework de React con servidor, rutas y renderizado incluidos.

Astro · Sitios de contenido muy rápidos.

Backend

Express · El framework web clásico y minimalista de Node.js.

NestJS · Backend estructurado, ideal para equipos grandes.

Node.js, Deno y Bun · Entornos para ejecutar JavaScript en el servidor.

Inteligencia artificial

SDK de Anthropic · Conecta tu app con Claude (TypeScript).

SDK de MCP · Crea servidores y clientes MCP.

AI SDK de Vercel · Chats y respuestas en tiempo real en apps web.

Calidad

TypeScript · Agrega tipos y detecta errores antes de ejecutar.

Vitest y Jest · Frameworks de pruebas.

ESLint · Revisa el estilo y los errores comunes.

Paquetes y construcción

npm · Gestor y registro de paquetes (también pnpm y Yarn).

Vite · Arma y sirve tu proyecto en desarrollo de forma muy rápida.

TAMAÑO DEL ECOSISTEMA

Es **el más grande y el que más rápido cambia**. Su ventaja: hay un paquete para casi todo. Su desafío: elegir bien. Prefiere librerías populares, con mantenimiento activo y buena documentación.

CONEXIÓN CON EL CURSO

JavaScript + IA

Dos caminos: usar la IA para **programar** en JavaScript y usar JavaScript para **construir apps con IA**.

La IA te ayuda a programar

JavaScript es de los lenguajes con más código público, así que asistentes como Claude lo dominan: crean componentes, APIs y pruebas.

JavaScript construye apps con IA

Con el **SDK oficial de Anthropic** (`npm install @anthropic-ai/sdk`) llamas a Claude, y con el **SDK de MCP** expones herramientas.

LIMITACIÓN REAL

El ecosistema cambia tan rápido que la IA puede mezclar `require` con `import`, APIs del navegador con las de Node.js o versiones antiguas de frameworks como React o Next.js. Revisa, ejecuta y prueba siempre.

Buenas prácticas al pedir código JavaScript

Di dónde corre

"Navegador" o "Node.js 24", y el framework con su versión.

Pide módulos ES

`import/export`, no `require`.

Considera TypeScript

Los tipos ayudan a la IA y a ti a detectar errores.

Comparte contexto

Pega el `package.json` y los archivos relacionados.

Prompt de ejemplo para copiar

```
// Prompt
Actúa como desarrollador senior de JavaScript con Node.js 24 LTS.
Crea una API REST con Express para gestionar Cursos (listar, crear,
eliminar) usando módulos ES (import/export) y async/await. Valida los datos
de entrada, agrega pruebas con Vitest y explícame cada archivo en una lista
de 5 puntos. No uses require ni APIs del navegador.
```

Flujo sugerido con IA

1 — Especificar
Describe qué debe hacer y sus reglas.

2 — Generar
Pide el código en pasos pequeños.

3 — Probar
Corre las pruebas con `npm test`.

4 — Revisar
Lee el código y entiende cada línea.

6 RESUMEN

Para recordar

JavaScript es el lenguaje de la web: el único que los navegadores ejecutan de forma nativa. Es dinámico, orientado a eventos y asíncrono gracias al *event loop*. Con **Node.js** también corre en servidores, y con React Native y Electron, en móviles y escritorio. Su ecosistema es enorme y cambia rápido. Apréndelo junto con TypeScript para construir productos web completos.

Web
Presente en casi el 99 % de los sitios web.

Asíncrono
Un solo hilo que nunca se queda esperando.

Full stack
Interfaz, servidor y móvil con un mismo lenguaje.

BIBLIOGRAFÍA

Fuentes consultadas · consultadas el 5 de noviembre de 2026

OFICIALES · ESTÁNDAR Y ENTORNOS

- Ecma International (jun. 2026). *ECMA-262, 17th edition: ECMAScript 2026 Language Specification*. ecma-international.org
- Node.js. *Node.js 26 release* y calendario de versiones. nodejs.org
- MDN Web Docs. *JavaScript*. developer.mozilla.org/es/docs/Web/JavaScript

IA Y SDK

- Anthropic. *TypeScript SDK*. platform.claude.com/docs
- Model Context Protocol. *TypeScript SDK*. modelcontextprotocol.io

RANKING, USO Y PRENSA

- W3Techs (nov. 2026). *Usage statistics of JavaScript as client-side programming language*. w3techs.com
- TIOBE Software (oct. 2026). *TIOBE Index* y *Hall of Fame*. tiobe.com/tiobe-index
- InfoWorld (2026). *ECMAScript 2026 specification approved*. infoworld.com
- FOSSlife. *JavaScript Tops List of Most Commonly Used Programming Languages* (encuesta Stack Overflow 2021). fosslife.org
- fireup.pro (oct. 2026). *Node.js 26 LTS: what's new*. fireup.pro

HISTORIA

- Wikipedia. *JavaScript* y *ECMAScript*. en.wikipedia.org

PARA SEGUIR APRENDIENDO

- Guía de JavaScript en español: developer.mozilla.org/es
- Aprende Node.js: nodejs.org/learn



MATERIAL ELABORADO POR

Felipe Cuevas C.

Instructor y autor del curso **Desarrollando con Inteligencia Artificial**. Ingeniero informático, desarrollador senior y docente universitario.

felipecuevas.dev

Material complementario para estudiantes · noviembre de 2026